

STRESZCZENIE

ABSTRACT

SPIS TREŚCI

Wykaz ważniejszych oznaczeń i skrótów	5
1. Wstęp i cel pracy	6
2. Wprowadzenie do dziedziny	7
2.1. Rynek gier przeglądarkowych	7
2.2. Rynek gier geolokalizacyjnych	7
2.3. Rodzaje gier konkurencyjnych	7
3. Wykorzystane technologie	8
3.1. Frontendowe (Filip Jezierski)	8
3.1.1. Progressive Web App	8
3.1.2. React	8
3.1.3. Axios	9
3.1.4. Leaflet	9
3.2. Backendowe (Michał Turski)	9
3.2.1. C# ASP .NET Core	9
3.2.2. Wykorzystane pakiety NuGet	9
3.3. Baza danych (Michał Turski)	10
4. Narzędzia	11
4.1. System kontroli wersji - Git i GitHub (Michał Pawiłojć)	11
4.1.1. Git	11
4.1.2. GitHub	11
4.2. Konteneryzacja - Docker (Michał Pawiłojć)	12
4.2.1. Czym jest konteneryzacja?	12
4.2.2. Docker	12
4.2.3. Docker Compose	12
4.2.4. Korzyści wynikające z konteneryzacji	12
4.2.5. Przemyslenia i ograniczenia	13
4.3. Zarządzanie projektem - YouTrack (Karina Wołoszyn)	13
4.4. Zintegrowane środowisko programistyczne - VSC, VS, Rider (Michał Pawiłojć)	14
4.4.1. Backend: Visual Studio i Rider	14
4.4.2. Frontend: Visual Studio Code	14
4.4.3. Wspólne praktyki i integracja IDE z projektem	14
4.5. Projektowanie interfejsu użytkownika - Figma (Karina Wołoszyn)	15
5. Analiza wymagań	16
5.1. Koncepcja gry	16
5.2. Przebieg gry	16
5.3. Tryby rozgrywki	16
5.4. Wymagania funkcjonalne	16
5.5. Wymagania нефункционалне (Filip Jezierski)	16
5.5.1. Wymagania jakościowe	16
5.5.2. Wymagania wydajnościowe	16

5.6. Wybór platformy (Filip Jezierski)	16
5.6.1. Natywna aplikacja mobilna	17
5.6.2. Aplikacja webowa PWA	17
5.7. Ograniczenia projektu (Filip Jezierski)	17
5.7.1. Środowisko aplikacji	17
5.7.2. Zdjęcia lokalizacji	17
5.8. Kryteria akceptacji (Filip Jezierski)	18
6. Ograniczenia projektu	19
7. Projekt systemu	20
7.1. Architektura systemu	20
7.2. Architektura logiki biznesowej	20
7.2.1. Wzorzec CQRS (Command Query Responsibility Segregation)	20
7.2.2. Zastosowanie DDD (Domain-Driven Design)	21
7.3. Baza danych	21
7.3.1. Diagram ERD	21
7.3.2. Opis głównych tabel i związków	21
7.4. Analiza interakcji użytkownika z systemem	23
7.4.1. Przypadki użycia aplikacji	24
7.4.2. Diagram sekwencji rozgrywki	24
7.5. Estetyka i design	25
7.5.1. Ogólny projekt interfejsu	25
7.5.2. Kolorystyka	25
7.5.3. Typografia	26
7.5.4. Ikony	26
7.5.5. Responsywność	26
8. Implementacja	28
9. Testowanie	29
10. Podsumowanie	30
11. Możliwość dalszego rozwoju	31
Wykaz literatury	32
Wykaz rysunków	32
Wykaz tabel	33

WYKAZ WAŻNIEJSZYCH OZNACZEŃ I SKRÓTÓW

LINQ (ang. Language Integrated Query) - zestawu technologii opartych na integracji możliwości tworzenia zapytań bezpośrednio z językiem C#

SQL (ang. Structured Query Language) - strukturalny język zapytań

JWT (ang. JSON Web Token) - standardowy format bezpiecznego przesyłania informacji jako obiekt JSON

JSON (ang. JavaScript Object Notation) - format wymiany danych oparty na składni języka JavaScript

REST (ang. Representational State Transfer) - styl architektoniczny wykorzystywany przy projektowaniu usług sieciowych

API (ang. Application Programming Interface) – zestaw reguł i protokołów określających sposób komunikacji oraz integracji pomiędzy różnymi elementami aplikacji.

VCS (ang. Version Control System) - system kontroli wersji

CI/CD (ang. Continuous Integration/Continuous Delivery lub Deployment) - zestaw praktyk tworzenia oprogramowania

UI (ang. User Interface) - interfejs użytkownika

UX (ang. User Experience) - doświadczenie użytkownika

ITS (ang. Issue Tracker System) - sposób zarządzania systemem, który odpowiada na zapytania wysyłane dowolną drogą.

DTO (ang. Data Transfer Object) - obiekt służący do przenoszenia danych między warstwami aplikacji

ORM (ang. Object-Relational Mapping) - technika mapowania danych między systemem obiektowym a relacyjną bazą danych

DDD (ang. Domain-Driven Design) - podejście do projektowania oprogramowania, które skupia się na odwzorowywaniu rzeczywistego świata w modelu domeny.

CQRS (ang. Command Query Responsibility Segregation) - wzorzec architektoniczny polegający na rozdzieleniu operacji modyfikujących stan systemu od operacji odczytujących dane.

ERD (ang. Entity-Relationship Diagram) - diagram związków encji, służący do modelowania struktury bazy danych

1. WSTĘP I CEL PRACY

Celem projektu jest stworzenie gry geograficzno-geolokalizacyjnej polegającej na odgadywaniu miejsc związanych z Politechniką Gdańską na podstawie ich zdjęć. Dodatkowo dostępny będzie tryb terenowy, w którym użytkownik musi fizycznie udać się w odgadywane miejsce. Aby zwiększyć stopień interakcji między użytkownikami, zaplanowaliśmy również udostępnienie publicznego rankingu, zachęcającego do poprawiania swoich wyników, a także dodanie trybu wieloosobowego, w którym gracze rywalizowaliby między sobą, ścigając się o to, kto pierwszy odgadnie zadaną lokalizację.

Aplikacja taka pozwalałaby na interaktywne poznawanie terenów kampusu dla szerokiego grona zainteresowanych, od osób niezwiązanych z uczelnią, do absolwentów oraz pracowników PG. Szczególnie przydatnym zastosowaniem byłoby zapoznanie przyszłych i nowych studentów z okolicami uczelni, ułatwiając im orientację w nowym otoczeniu.

Gry przeglądarkowe cieszyły się popularnością od momentu ich powstania. Ich głównym atutem są niskie wymagania sprzętowe czy brak potrzeby instalacji, które od początku poszerzyły grono potencjalnych odbiorców. Dodatkowo, użytkownicy za pośrednictwem dostępu do internetu mogą wchodzić ze sobą w interakcje i rywalizować dzięki udostępnianym rankingom.

Popularność gier komputerowych znacząco wzrosła podczas pandemii COVID-19, gdzie przez potrzebę pozostania w izolacji możliwości spędzania wolnego czasu były ograniczone. Wydłużony czas pozostania w domach przyczynił się też do wzrostu zainteresowania grami geograficznymi i geolokalizacyjnymi. Jednym z czołowych produktów tego zjawiska został GeoGuessr, który mimo wypuszczenia na rynek 6 lat wcześniej, został spopularyzowany dopiero w 2020 roku. Sukces ten można przypisać idei połączenia rozrywki i odkrywania świata z wykorzystaniem Google Street View, dzięki której użytkownicy mogą się wykazać wiedzą na temat architektury, ukształtowania terenu oraz panującego w danym kraju języka.

Podobnym trendem wykazały się aplikacje mobilne, które ze względu na szybki rozwój urządzeń przenośnych, pozwalają graczowi połączyć się z dowolnego miejsca, w dowolnej chwili. Zlikwidowały one potrzebę posiadania drogiego i nieporęcznego sprzętu, stawiając na wygodę użytkownika. Atuty te wykorzystwała gra Pokemon Go, która z pomocą dobrze znanej marki podbiła rynek, osiągając pułap 500 milionów pobrań w pierwszym roku po debiucie. Gra ta korzysta z funkcji GPS do znalezienia wirtualnych aren rozsianych po prawdziwych lokalizacjach, przyczyniając się do zwiększenia immersji rozrywki.

Po dokonaniu powyższej analizy rynku, postanowiliśmy w ramach projektu inżynierskiego stworzyć aplikację łączącą w sobie dotychczas wymienione aspekty.

2. WPROWADZENIE DO DZIEDZINY

2.1. Rynek gier przeglądarkowych

2.2. Rynek gier geolokalizacyjnych

2.3. Rodzaje gier konkurencyjnych

3. WYKORZYSTANE TECHNOLOGIE

3.1. Frontendowe (Filip Jezierski)

Do implementacji frontendu zastosowano bibliotekę **React.js** z **TypeScriptem**, umożliwiającą tworzenie aplikacji w technologii **Progressive Web App**. Komunikację z backendem przez REST API zapewnia biblioteka **Axios**. Do obsługi map i lokalizacji użyto biblioteki **Leaflet**.

3.1.1. Progressive Web App

Progressive Web App (PWA) to podejście do tworzenia aplikacji webowych, które umożliwia działanie podobne do natywnej aplikacji mobilnej lub desktopowej. PWA można zarówno używać w przeglądarce, jak i zainstalować jako niezależną od przeglądarki aplikację na urządzeniu. Technologia ta zapewnia też działanie podstawowych funkcji aplikacji bez dostępu do internetu.

Publikowanie PWA przez platformy dystrybucji cyfrowej takie jak Apple App Store czy Google Play jest możliwe, lecz opcjonalna - użytkownik może również zainstalować aplikację bezpośrednio z poziomu przeglądarki internetowej, wybierając opcję "Dodaj do ekranu głównego".

Aplikacja webowa musi spełniać poniżej wymienione wymagania aby mogła być zainstalowana na urządzeniu jako PWA:

- **Bezpieczne źródło** - aplikacja musi być serwowana przez HTTPS, aby zapewnić bezpieczeństwo i autentyczność danych.
- **Service Worker** - dzięki zastosowaniu tego mechanizmu aplikacja może ładować do pamięci podręcznej fragmenty, które jeszcze nie zostały użyte, co sprawia że można ją później uruchomić w trybie offline.
- **Plik manifestu** - w aplikacji powinien być zawarty plik JSON zawierający następujące informacje: name lub short_name, zestaw ikon (w szczególności w rozdzielczościach 192 px i 512 px), start_url oraz parametr display (z jedną z wartości: fullscreen, standalone, minimal-ui lub window-controls-overlay).

3.1.2. React

React to popularna biblioteka JavaScript stworzona przez Facebooka, służąca do budowy nowoczesnych interfejsów użytkownika w oparciu o komponenty. Dzięki wykorzystaniu wirtualnego DOM-u React umożliwia szybkie i wydajne aktualizacje widoku bez potrzeby bezpośrednich manipulacji w strukturze HTML.

Architektura komponentowa pozwala na dzielenie aplikacji na niezależne, łatwo testowalne i ponownie wykorzystywane elementy. W projekcie wykorzystano komponenty funkcyjne oraz hooki (useState, useEffect) do zarządzania stanem i cyklem życia komponentów.

React zapewnia dużą elastyczność oraz możliwość integracji z innymi bibliotekami frontendowymi. Do komunikacji z backendem wykorzystano funkcje asynchroniczne zdefiniowane w specjalnie wydzielonych serwisach. Dodatkowo, React umożliwia tworzenie aplikacji responsywnych, co było istotne w kontekście zapewnienia poprawnego działania na różnych typach urządzeń.

W projekcie zdecydowaliśmy się również na użycie języka TypeScript - nadzbioru JavaScriptu, który wprowadza statyczne typowanie. TypeScript ułatwia wczesne wykrywanie błędów, usprawnia podpowie-

dzi edytora oraz poprawia czytelność i bezpieczeństwo kodu. Dodatkowo umożliwia łatwe modelowanie danych przesyłanych między komponentami oraz poprzez API.

3.1.3. *Axios*

Axios to biblioteka JavaScript służąca do wykonywania asynchronicznych zapytań HTTP. W projekcie została użyta do komunikacji pomiędzy frontendem i backendem poprzez REST API.

W porównaniu do natywnej funkcji `fetch`, Axios oferuje uproszczoną składnię, automatyczne przetwarzanie odpowiedzi w formacie JSON oraz bardziej zaawansowane możliwości konfiguracji — takie jak ustawianie domyślnych nagłówek, bazowego URL, timeoutów i łatwiejsze przechwytywanie błędów. W połączeniu z TypeScriptem, Axios umożliwia również bardziej precyzyjne typowanie odpowiedzi serwera.

3.1.4. *Leaflet*

Leaflet to otwartoźródłowa biblioteka JavaScript, służąca do tworzenia interaktywnych map. W projekcie użyliśmy jej w połączeniu z React Leaflet - wrapperem, który udostępnia funkcjonalność Leafleta w postaci gotowych komponentów React.

Biblioteka Leaflet umożliwia wyświetlanie map z zewnętrznych źródeł (np. OpenStreetMap) oraz nanoszenie na nie dodatkowych elementów, takich jak znaczniki, warstwy czy lokalizacja użytkownika z wykorzystaniem geolokalizacji przeglądarki. Dodatkowo umożliwia wykrywanie interakcji z mapą, np. kliknięć, oraz zwracanie odpowiadających im współrzędnych geograficznych.

3.2. *Backendowe (Michał Turski)*

W backendzie zastosowano język **C#** z frameworkiem **.NET** oraz paczki NuGet w celu stworzenia aplikacji serwerowej realizującej wymagania biznesowe i obsługującej komunikację z bazą danych **PostgreSQL**.

3.2.1. *C# ASP .NET Core*

ASP .NET Core od lat utrzymuje się w czołówce najchętniej wybieranych frameworków do tworzenia aplikacji webowych i API. Jego popularność wynika przede wszystkim z wysokiej modularności oraz niezwyklej elastyczności w adaptowaniu się do różnych scenariuszy projektowych.

Atutem ASP.NET Core jest jego pełna wieloplatformowość — aplikacje mogą być uruchamiane na systemach Windows, Linux oraz macOS bez konieczności wprowadzania zmian w kodzie. To znacząco ułatwia wdrażanie aplikacji w różnorodnych środowiskach, w tym także w kontenerach Docker czy chmurze. Aktualnie w trakcie procesu wytwarzania oprogramowania wykorzystywany jest system Windows, WSL, a także aplikacja odpalana jest w kontenerze Dockerowym.

Ponadto środowisko to w znacznym stopniu wspiera rozwój oprogramowania zgodnie z zasadami SOLID. Dzięki wbudowanemu mechanizmowi wstrzykiwania zależności (dependency injection), ułatwione jest tworzenie spójnych i jednoznacznie zdefiniowanych interfejsów, a także separacja odpowiedzialności pomiędzy warstwami aplikacji.

3.2.2. *Wykorzystane pakiety NuGet*

W projekcie wykorzystano szereg pakietów NuGet, które usprawniają implementację kluczowych funkcjonalności oraz wspierają dobre praktyki projektowe. Pakiety NuGet to zewnętrzne biblioteki, które

można łatwo zintegrować z projektem .NET w celu rozszerzenia jego możliwości. Umożliwiają one ponowne wykorzystanie sprawdzonego kodu oraz automatyzację zarządzania zależnościami, co znacząco przyspiesza proces tworzenia i utrzymania aplikacji.

Wykorzystano następujące pakiety NuGet w aplikacji wraz z krótkim opisem:

- **AutoMapper** – Ułatwia przekształcanie danych pomiędzy klasami, co jest szczególnie przydatne przy mapowaniu obiektów domenowych na klasy DTO (Data Transfer Object) oraz odwrotnie. Redukuje potrzebę ręcznego kopiowania właściwości, co przekłada się na większą czytelność i spójność kodu.
- **FluentValidation** – Usprawnia proces walidacji danych, wprowadzając dedykowaną warstwę odpowiedzialną za sprawdzanie poprawności obiektów wejściowych. Pozwala na definiowanie reguł walidacyjnych w sposób deklaratywny, oddzielając je od logiki aplikacji.
- **Microsoft.EntityFrameworkCore** – Biblioteka ORM umożliwiająca odwzorowywanie klas C# na struktury relacyjnej bazy danych. Umożliwia pracę z bazą danych z użyciem LINQ oraz migracji, eliminując potrzebę pisania zapytań SQL.
- **Npgsql.EntityFrameworkCore.PostgreSQL** – Dostarcza integrację Entity Framework Core z bazą danych PostgreSQL, umożliwiając pełne wsparcie dla tej platformy w środowisku .NET.
- **Swashbuckle.AspNetCore.Swagger** – Narzędzie generujące dokumentację API w formacie OpenAPI (Swagger). Umożliwia wygodne testowanie endpointów poprzez interaktywny interfejs webowy oraz automatyczne tworzenie opisów metod HTTP.
- **Microsoft.AspNetCore.Authentication.JwtBearer** – Zapewnia obsługę uwierzytelniania z użyciem tokenów JWT (JSON Web Token), które są powszechnym rozwiązaniem przy autoryzacji użytkowników w aplikacjach webowych typu REST API.
- **xUnit** – Framework służący do tworzenia testów jednostkowych, umożliwiający automatyczne sprawdzanie poprawności działania kodu aplikacji.
- **Moq** – Biblioteka wspierająca testowanie poprzez łatwe tworzenie atrap (mocków) i symulowanie zachowań obiektów, co pozwala izolować testowane elementy od zależności.
-
- **MediatR** - Biblioteka implementująca wzorzec Mediator, który ułatwia komunikację między komponentami aplikacji poprzez pośrednika. Pomaga uporządkować przepływ komunikacji i usunąć bezpośrednie zależności między warstwami lub klasami.

3.3. Baza danych (Michał Turski)

W projekcie zastosowano relacyjną bazę danych **PostgreSQL**, która jest darmowym i otwartym oprogramowaniem typu **Open Source**. Jest to szczególnie korzystne w przypadku projektów niekomercyjnych.

Baza danych wyróżnia się zgodnością ze standardem SQL oraz obsługą zaawansowanych typów danych, takich jak **JSONB**, umożliwiając efektywne łączenie struktur relacyjnych z nierelacyjnymi.

Do zarządzania bazą danych wykorzystano narzędzia administracyjne, takie jak **pgAdmin** i **DB-aver**, oferujące graficzne interfejsy ułatwiające pracę z bazą danych.

4. NARZĘDZIA

4.1. System kontroli wersji - Git i GitHub (Michał Pawiłojć)

W trakcie realizacji projektu *UniGuesser* kluczową rolę odegrało zastosowanie systemu kontroli wersji Git oraz platformy GitHub. Ich wykorzystanie umożliwiło efektywne zarządzanie kodem źródłowym, koordynację pracy zespołu oraz utrzymanie przejrzystości i historii zmian w projekcie.

4.1.1. Git

Git to rozproszony system kontroli wersji VCS(ang. *Version Control System*), stworzony przez Linusa Torvaldsa w 2005 roku. Jego podstawowym zadaniem jest śledzenie zmian w plikach projektu, umożliwienie pracy wielu programistów nad tym samym kodem oraz szybkie cofanie się do wcześniejszych wersji w razie potrzeby. Git opiera się na repozytorium lokalnym i zdalnym, co zapewnia dużą elastyczność pracy – możliwa jest nawet praca offline z późniejszą synchronizacją.

Do podstawowych operacji należą:

- `git init` – inicjalizacja nowego repozytorium,
- `git add` – dodanie zmian do indeksu (staging area),
- `git commit` – zapisanie zmian w historii projektu,
- `git branch` – zarządzanie gałęziami (np. tworzenie odgałęzień na nowe funkcjonalności),
- `git merge` – scalanie gałęzi,
- `git log` – przegląd historii commitów.

Stosowanie Gita pozwala na łatwe śledzenie błędów, eksperymentowanie z nowymi funkcjonalnościami bez wpływu na główny kod oraz pracę równoległą wielu osób. W przypadku zmian powodujących błąd powrót do działającej wersji jest praktycznie bezkosztowy.

4.1.2. GitHub

GitHub to jedna z najpopularniejszych platform umożliwiających przechowywanie zdalnych repozytoriów Git. Oferuje dodatkowe funkcje, takie jak:

- pull requesty (prośby o połączenie zmian z główną gałęzią projektu),
- code review (wzajemna kontrola kodu w zespole),
- zarządzanie zgłoszeniami (issues),
- integracja z narzędziami CI/CD (np. GitHub Actions).

W projekcie *UniGuesser* GitHub pełnił rolę centralnego punktu współpracy, umożliwiając m.in. zgłaszanie błędów, przeglądanie zmian oraz automatyczne uruchamianie testów jednostkowych i wdrożeń (CI/CD).

Repozytorium projektu zostało zorganizowane w sposób modularny – z podziałem na frontend, backend i konfigurację Docker. Każdy członek zespołu pracował na oddzielnych branchach, a zmiany były scalane po przejściu procesu *code review*. Takie podejście zapewniło stabilność projektu i ułatwiło jego rozwój.

4.2. Konteneryzacja - Docker (Michał Pawłojć)

Współczesne aplikacje webowe, takie jak *UniGuesser*, składają się z wielu komponentów - frontend, backend, baza danych - muszą one współdziałać w spójnym środowisku. Tradycyjna metoda instalowania zależności lokalnie prowadzi do problemów ze zgodnością, trudnością odtworzenia środowiska oraz błędami zależnymi od platformy. Rozwiązaniem tego problemu jest konteneryzacja - technika, która pozwala zbudować przenośne, spójne środowisko dla każdego komponentu aplikacji.

4.2.1. Czym jest konteneryzacja?

Konteneryzacja to proces pakowania aplikacji wraz z całym jej środowiskiem uruchomieniowym (zależnościami, bibliotekami, konfiguracją, itd.) do jednego, samodzielnego pakietu zwanego **kontenerem**. Taki kontener może być uruchomiony na dowolnym systemie operacyjnym, który obsługuje kontenery, bez potrzeby rekonfiguracji lub instalacji dodatkowego oprogramowania.

Kontenery są lekką alternatywą dla maszyn wirtualnych. W odróżnieniu od VM-ek, nie zawierają pełnego systemu operacyjnego, a współdzielą jądro systemu hosta. Dzięki temu zużywają mniej zasobów, szybciej się uruchamiają i łatwiej się nimi zarządza.

Dzięki konteneryzacji możliwe jest podejście „*build once, run anywhere*” - czyli raz zbudowany obraz aplikacji może działać tak samo na komputerze deweloperskim, serwerze testowym i w środowisku produkcyjnym.

4.2.2. Docker

W naszym projekcie wykorzystaliśmy **Dockera** — najpopularniejsze narzędzie do konteneryzacji. Docker umożliwia budowanie własnych obrazów aplikacji za pomocą plików konfiguracyjnych *Dockerfile*, a także zarządzanie kontenerami lokalnie lub w chmurze.

4.2.3. Docker Compose

Projekt *UniGuesser* składa się z kilku współdziałających usług:

- Frontend (React),
- Backend (.NET),
- Baza danych (PostgreSQL).

Do zarządzania tymi usługami wykorzystano **Docker Compose**. Umożliwia ono deklaratywne opisanie całego środowiska w jednym pliku YAML. W ten sposób tworzona jest wewnętrzna sieć, w której poszczególne usługi są w stanie się komunikować.

Całe środowisko można uruchomić jedną komendą:

```
docker-compose up --build
```

4.2.4. Korzyści wynikające z konteneryzacji

Użycie Dockera i Composa przyniosło wiele korzyści:

- **Powtarzalność środowiska** — każdy członek zespołu uruchamiał projekt dokładnie w taki sam sposób.
- **Izolacja komponentów** — każda usługa działała w swoim kontenerze.
- **Łatwe wdrażanie** — obrazy Dockera działały tak samo lokalnie, jak i na produkcji.

- **Integracja z CI/CD** — obrazy mogły być budowane i testowane w GitHub Actions.

4.2.5. *Przemyślenia i ograniczenia*

Choć konteneryzacja znacząco uprościła zarządzanie środowiskiem, napotkano również pewne trudności:

- **Krzywa uczenia** — wymagana była znajomość Dockera i Composa.
- **Wydajność na Windowsie** — niektóre operacje były wolniejsze w WSL2.
- **Debugowanie** — błędy konfiguracyjne były trudniejsze do wychwycenia.

Pomimo tych wyzwań, konteneryzacja odegrała kluczową rolę w sukcesie projektu.

4.3. *Zarządzanie projektem - YouTrack (Karina Wołoszyn)*

Do zarządzania projektem wykorzystane został Youtrack, który jest narzędziem typu ITS (ang. Issue Tracker System) stworzonym przez firmę JetBrains. Opiera się on na technologii Ajax i wspiera szeroki zakres metodyk zarządzania projektem, w tym podejść zwinnych, które również zostało wykonane do realizacji tego projektu.

System Youtrack umożliwia m.in. planowanie i organizację zadań, przypisywanie ich do konkretnych członków, ustalanie priorytetów, śledzenia postępów prac, dodawanie tagów, załączników czy tworzenie raportów. Dzięki rozbudowanym możliwościom filtrowania, szacowania i określania ilości spędzonego czasu, personalizacji widoków możliwe było bieżące monitorowanie i reagowanie na pojawiające się ewentualne problemy.

W ramach projektu przyjęto podejście oparte na zmodyfikowanej wersji metodyki Kanban, dostosowanej do potrzeb zespołu i charakteru realizowanego zadania. Tablica Kanban została podzielona na sześć głównych etapów:

- **Backlog** - przeznaczony do przechowywania wszystkich potrzebnych zadań do zrealizowania projektu. Te zadania obejmują duży zakres obowiązków, które są w następnym etapie rozdzielane na mniejsze, bardziej precyzyjne zadania, jeżeli są wybrane do realizacji w najbliższym sprincie.
- **Backlog sprintu** - zbierane są tu zadania do bieżącego sprintu. Dokładnie jest określany zakres wymagań, który musi spełnić, aby w pełni je zrealizować.
- **Wykonanie** - aktualnie wykonywane zadania. Przydzielone są one do konkretnego członka zespołu i mają ustawiony priorytet oraz liczbę ustalonych story points'ów.
- **Testowanie** - zbierane są tutaj wykonane zadania oczekujące na przetestowanie przez innego członka zespołu albo osobę wykonującą dane zadanie. W przypadku błędów lub niespełnienia wymagań, zadanie zostaje zwrócone do poprzedniego etapu w celu wprowadzenia poprawek.
- **Recenzja** - wykonane zadania oczekują na recenzję przez innego członka zespołu niż ten, który go wykonywał. Pozwala to na lepszą identyfikację błędów lub potencjalnych błędów. Podobnie jak w przypadku testowania, w przypadku niespełnienia kryteriów zadanie wraca do etapu 'Wykonanie'.
- **Zrobione** - przeznaczony do przechowywania wykonanych, przetestowanych i zrecenzowanych zadań.

4.4. Zintegrowane środowisko programistyczne - VSC, VS, Rider (Michał Pawłojć)

W trakcie realizacji projektu wykorzystano różne zintegrowane środowiska programistyczne (IDE), dostosowane do technologii używanych w poszczególnych komponentach aplikacji. Odpowiedni dobór narzędzi znacząco przyczynił się do efektywnej pracy zespołu, ułatwiając zarówno pisanie kodu, jak i jego debugowanie, testowanie oraz integrację z systemem kontroli wersji Git.

4.4.1. Backend: Visual Studio i Rider

Głównym środowiskiem wykorzystywanym do pracy nad backendem był **Visual Studio**, w wersji Community 2022, z uwagi na jego doskonałe wsparcie dla platformy .NET i języka C#.

Visual Studio oferowało pełny zestaw funkcjonalności ułatwiających pracę z aplikacjami ASP.NET Core, m.in.:

- integrację z systemem projektów .NET,
- intuicyjne debugowanie,
- wizualizację struktur danych,
- automatyczne wykrywanie problemów w kodzie (analiza statyczna),
- możliwość tworzenia i uruchamiania testów jednostkowych,
- integrację z narzędziami do konteneryzacji, np. Docker Tools.

Niektórzy członkowie zespołu korzystali również z alternatywnego IDE: **JetBrains Rider**. Był on preferowany w szczególności przez osoby przyzwyczajone do narzędzi JetBrains oraz pracujące głównie w środowisku Linux lub WSL. Rider zapewniał szybkie działanie, inteligentne podpowiedzi (ang. *code completion*) i integrację z systemami bazodanowymi oraz Dockerem.

Oba środowiska dobrze współpracowały z systemem kontroli wersji Git oraz pozwalały na łatwą konfigurację środowisk uruchomieniowych (launch profiles), co było kluczowe przy testowaniu aplikacji w kontenerach.

4.4.2. Frontend: Visual Studio Code

Do pracy nad frontendem, który został zaimplementowany w technologii **React** z wykorzystaniem **TypeScript**, wykorzystywano głównie **Visual Studio Code** (VS Code). Jest to lekkie, ale bardzo elastyczne narzędzie, które doskonale sprawdza się w projektach opartych na JavaScript, dzięki bogatemu ekosystemowi rozszerzeń.

VS Code ułatwiał szybkie uruchamianie aplikacji frontendowej za pomocą skryptów npm, a także pozwalał na debugowanie kodu przeglądarkowego poprzez wbudowany debugger.

4.4.3. Wspólne praktyki i integracja IDE z projektem

Pomimo stosowania różnych środowisk, zespół utrzymywał spójne praktyki pracy:

- **Standaryzacja formatowania kodu** — z wykorzystaniem plików konfiguracyjnych (`.editorconfig`, `.prettierrc`),
- **Wspólne launch profile** — umożliwiające uruchamianie backendu lokalnie lub w kontenerze,
- **Integracja z GitHubem** — dzięki której możliwa była praca zdalna, pull requesty i code review bezpośrednio z poziomu IDE,
- **Debugowanie lokalne i w kontenerze** — możliwe dzięki odpowiednim rozszerzeniom i konfiguracji,

- **Wsparcie dla Docker Compose** — uruchamianie całego środowiska aplikacji z poziomu IDE.

Różnorodność wykorzystywanych środowisk nie stanowiła problemu dzięki modularnej architekturze projektu oraz konteneryzacji, która gwarantowała, że kod backendu i frontend działał w identycznym środowisku niezależnie od lokalnych ustawień IDE.

Zastosowanie odpowiednich IDE w zależności od warstwy aplikacji pozwoliło zwiększyć produktywność zespołu. Visual Studio oraz Rider znakomicie sprawdziły się przy rozwoju backendu w .NET, natomiast VS Code był naturalnym wyborem do pracy nad frontendem w React. Elastyczność i rozszerzalność tych środowisk umożliwiła sprawną pracę zespołową oraz efektywne wdrażanie zmian w całym cyklu życia aplikacji.

4.5. *Projektowanie interfejsu użytkownika - Figma (Karina Wołoszyn)*

W celu zbudowania wspólnej i klarownej wizji interfejsu użytkownika, została wykorzystana Figma, czyli aplikacja webowa pozwalająca na projektowanie prototypów UI/UX (ang. User Interface/User Experience). Narzędzie to umożliwia pracę zespołową w czasie rzeczywistym, czyniąc komunikację i wprowadzanie szybszą, i prostszą. W projekcie *UniGuesser* szczególny nacisk położono na minimalizm i prostotę zaprojektowanego interfejsu. Jednocześnie uwzględniono Księgę Identyfikacji Wizualnej Politechniki Gdańskiej, aby właściwie odwzorować kolorystykę oraz rozmieszczenie elementów zgodnie z przyjętymi standardami. Figma posiada szeroki wachlarz funkcjonalności, które w dużym stopniu usprawniają pracę zespołu. Do najważniejszych z nich należą:

- **Zarządzanie warstwami i grupami** - pozwala na precyzyjną organizację elementów, co jest szczególnie ważne przy tworzeniu skomplikowanej aplikacji, aby zachować jej spójność wizualną.
- **Tworzenie komponentów i wariantów** - umożliwia tworzenie wielu wariantów wykorzystywanych elementów UI, co pozwala na testowanie różnych rozwiązań bez konieczności zmiany kodu.
- **Integracje z narzędziami deweloperskimi** - wspierają płynne przejście od fazy projektowej do wdrożenia. Jednym z takich narzędzi jest GitHub, który oferuje łatwą konfigurację, możliwość dodawania zgłoszeń oraz pull requestów, wspomagając kontrolę wersji całego projektu.
- **Eksport zasobów i generowanie fragmentów kodu CSS** - przyspiesza implementację zaprojektowanych widoków.
- **Biblioteki stylów i zasobów** - pozwala na utrzymanie spójności wizualnej przechowując zestaw fontów, kolorów, efektów i rozmiarów elementów dostępnych dla całego zespołu.

Dzięki tym zaawansowanym funkcjom Figma stała się kluczowym elementem w realizacji procesu projektowania interfejsu użytkownika *UniGuesser*.

5. ANALIZA WYMAGAŃ

5.1. *Koncepcja gry*

5.2. *Przebieg gry*

5.3. *Tryby rozgrywki*

5.4. *Wymagania funkcjonalne*

5.5. *Wymagania нефunkcjonalne (Filip Jezierski)*

Wymagania нефunkcjonalne określają kryteria jakościowe, które aplikacja powinna spełniać, aby zapewnić odpowiednie doświadczenie użytkownika oraz efektywne działanie systemu. W kontekście opracowywanej aplikacji mobilnej, kluczowe wymagania нефunkcjonalne obejmują aspekty takie jak wydajność, bezpieczeństwo oraz użyteczność.

5.5.1. *Wymagania jakościowe*

- obsługa różnych rozdzielczości ekranów i urządzeń mobilnych,
- dostępność aplikacji na popularnych platformach mobilnych (Android, iOS), a także jako aplikacja webowa,
- intuicyjny i responsywny interfejs użytkownika,
- aplikacja powinna w odpowiedni sposób obsługiwać błędy i wyjątki, takie jak nieprawidłowe dane wejściowe czy brak internetu, zapewniając stabilność działania,
- ochrona danych użytkowników poprzez implementację odpowiednich mechanizmów bezpieczeństwa - szyfrowanie danych, bezpieczne logowanie, ochrona przed atakami typu SQL Injection i XSS.

5.5.2. *Wymagania wydajnościowe*

- szybkie (<2 s) ładowanie aplikacji i minimalny czas oczekiwania na reakcję interfejsu użytkownika,
- ładowanie zdjęć lokalizacji w czasie nie dłuższym niż 3 sekundy,
- szybkie (<10 s) ładowanie danych geolokalizacyjnych zapewniające płynne działanie trybu gry opartego na lokalizacji,

5.6. *Wybór platformy (Filip Jezierski)*

Wybór odpowiedniej platformy dla opracowywanej aplikacji mobilnej jest kluczowym elementem procesu projektowego, mającym istotny wpływ na jej funkcjonalność, dostępność oraz doświadczenie użytkownika, a także łatwość wdrożenia i utrzymania. Na etapie analizy wymagań podjęto decyzję o stworzeniu dwóch testowych wersji aplikacji: aplikacji webowej typu PWA (Progressive Web App)

oraz natywnej aplikacji mobilnej wykorzystującej technologię .NET MAUI (Multi-platform App UI). Obie platformy oferują unikalne korzyści i wyzwania, które zostały dokładnie rozważone w kontekście specyfiki projektu.

5.6.1. Natywna aplikacja mobilna

Aplikacje natywne są tworzone z wykorzystaniem dedykowanych narzędzi i języków programowania przeznaczonych dla konkretnych systemów operacyjnych, takich jak Android czy iOS. W przypadku technologii .NET MAUI możliwe jest tworzenie aplikacji wieloplatformowych przy zachowaniu wspólnej bazy kodu, co znacząco skraca czas implementacji i ułatwia utrzymanie.

Jednakże proces publikacji aplikacji natywnych w sklepach takich jak Google Play czy Apple App Store wiąże się z dodatkowymi wymaganiami i procedurami zatwierdzania, a także z koniecznością budowania i testowania aplikacji na różne systemy operacyjne, co może wydłużyć czas wdrożenia.

5.6.2. Aplikacja webowa PWA

Aplikacja webowa typu PWA (Progressive Web App) łączy cechy klasycznej strony internetowej z funkcjonalnością aplikacji mobilnej. Dzięki wykorzystaniu nowoczesnych technologii webowych, możliwe jest stworzenie aplikacji działającej w trybie offline, instalowanej bezpośrednio z przeglądarki oraz zapewniającej wrażenia zbliżone do aplikacji natywnej.

Ze względu na łatwość dostępu (poprzez przeglądarkę internetową) oraz brak konieczności przechodzenia przez proces zatwierdzania w sklepach aplikacji, PWA stanowi atrakcyjną opcję dla naszego projektu. Jednakże, pewne ograniczenia związane z dostępem do funkcji urządzenia (takich jak geolokalizacja) oraz wydajnością w porównaniu do aplikacji natywnych muszą być uwzględnione podczas projektowania i implementacji.

5.7. Ograniczenia projektu (Filip Jezierski)

Ograniczenia projektu wynikają przede wszystkim z przyjętych założeń technologicznych oraz charakteru opracowywanego rozwiązania. Kluczowe znaczenie mają tu specyfika aplikacji webowej typu PWA, zastosowanie mechanizmów geolokalizacji jako podstawowego elementu funkcjonalnego, a także decyzje dotyczące sposobu pozyskiwania i wykorzystania materiałów wizualnych. Wymienione czynniki w naturalny sposób wpływają na zakres możliwości implementacyjnych, wydajność rozwiązania oraz doświadczenie użytkownika końcowego.

5.7.1. Środowisko aplikacji

Aplikacja będzie udostępniana jako aplikacja webowa, dostępna przez przeglądarkę internetową, z możliwością instalacji na ekranie głównym urządzenia. Funkcjonalność trybu gry wykorzystującego geolokalizację będzie dostępna wyłącznie na urządzeniach mobilnych, które wspierają tę technologię. W przypadku urządzeń stacjonarnych, nawet jeśli geolokalizacja jest technicznie możliwa, jej dokładność jest niewystarczająca do zapewnienia poprawnego działania tej funkcji.

5.7.2. Zdjęcia lokalizacji

W projekcie zrezygnowano z wykorzystywania zdjęć 360° pochodzących z zewnętrznych źródeł, takich jak interfejsy API udostępniane przez firmy trzecie. Decyzję tę podjęto z uwagi na chęć uniknięcia potencjalnych problemów związanych z licencjonowaniem, ograniczoną dostępnością materiałów oraz

zróżnicowaną jakością zdjęć. Dodatkowo korzystanie z takich źródeł mogłoby wprowadzać ograniczenia dotyczące liczby i różnorodności dostępnych lokalizacji.

W celu uproszczenia procesu pozyskiwania materiałów wizualnych oraz umożliwienia użytkownikom współtworzenia bazy lokalizacji, aplikacja będzie wykorzystywać standardowe zdjęcia dwuwymiarowe. Tego typu fotografie nie wymagają specjalistycznego sprzętu i mogą być wykonywane przy użyciu dowolnego smartfona. Użytkownicy będą mieli możliwość przesyłania własnych zdjęć lokalizacji, które po weryfikacji przez moderatorów zostaną dodane do bazy danych aplikacji.

Takie rozwiązanie zapewnia pełną kontrolę nad jakością i zgodnością materiałów z założeniami projektu, a jednocześnie pozwala na stopniowe rozszerzanie bazy lokalizacji o nowe zdjęcia, dostarczane zarówno przez zespół projektowy, jak i społeczność użytkowników.

5.8. Kryteria akceptacji (Filip Jezierski)

Ogólne kryteria:

- Aplikacja działa na urządzeniach mobilnych z systemami Android i iOS, na komputerach stacjonarnych oraz w przeglądarkach internetowych
- Intuicyjny interfejs użytkownika umożliwiający łatwe korzystanie z aplikacji
- Responsywny design dostosowujący się do różnych rozmiarów ekranów, w szczególności urządzeń mobilnych
- Stabilne działanie aplikacji bez krytycznych błędów i awarii
- Testy manualne wszystkich zaplanowanych funkcjonalności przeprowadzone na wszystkich wspieranych platformach

Zaimplementowane funkcjonalności:

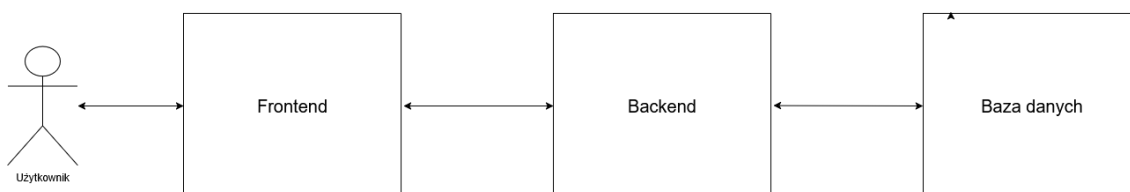
- Rejestracja i logowanie użytkowników
- Wybór trybu rozgrywki i poziomu trudności
- Dwa tryby gry:
 - odgadywanie lokalizacji na podstawie zdjęcia poprzez wskazanie jej na mapie
 - odnajdywanie lokalizacji na podstawie zdjęcia i potwierdzanie obecności poprzez geolokalizację
- Możliwość przerywania i kontynuowania rozgrywki
- Wyświetlanie podsumowania wyników każdej rundy gry
- Historia rozgrywek dostępna w profilu użytkownika
- Ogólnodostępny ranking graczy
- Dodawanie lokalizacji przez użytkowników (po weryfikacji przez moderatorów)

6. OGRANICZENIA PROJEKTU

7. PROJEKT SYSTEMU

7.1. Architektura systemu

W aplikacji zastosowano architekturę warstwową, której szczegółowa implementacja zostanie opisana w niniejszym rozdziale. Na najwyższym poziomie abstrakcji system można podzielić na trzy główne warstwy: warstwę prezentacji (frontend), warstwę logiki biznesowej (backend) oraz warstwę persystencji danych (baza danych). Podział ten odzwierciedla również strukturę kontenerów Dockera wykorzystywanych przy wdrożeniu aplikacji.



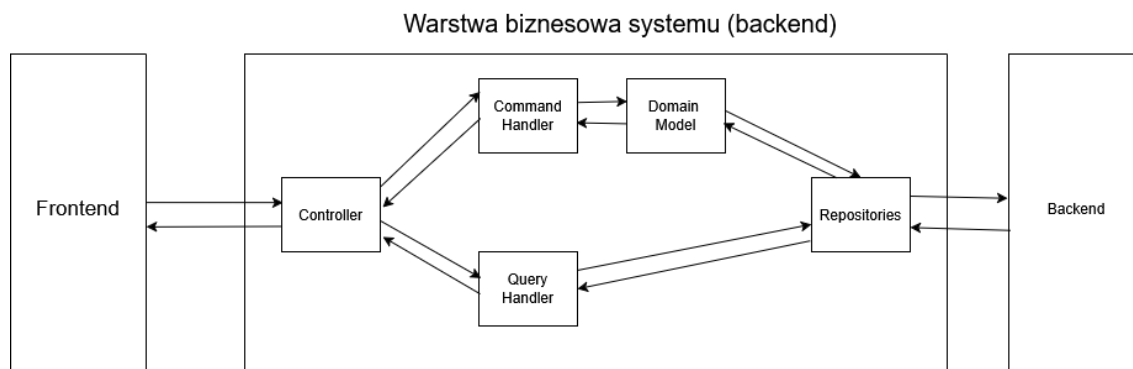
Rysunek 7.1: Uproszczony schemat architektury systemu UniGuesser – ogólny podział na warstwy

7.2. Architektura logiki biznesowej

Architektura w aplikacji **UniGuesser** została zbudowana w oparciu o Command-Query Responsibility Segregation (CQRS) oraz Domain-Driven Design (DDD), które są obecnie standardem w wytwarzaniu oprogramowania.

7.2.1. Wzorzec CQRS (Command Query Responsibility Segregation)

Wykorzystany został również wzorzec CQRS. Zakłada on rozdzielenie operacji modyfikujących stan systemu (komendy) od operacji, które odczytują dane (zapytania). Pozwala to na lepsze skalowanie i rozdzielenie logiki aplikacji. Dzięki temu można optymalizować każdą z tych operacji niezależnie, co przekłada się na wydajność i czytelność kodu. Należy również zwrócić uwagę, że nasza aplikacja posiada mały ruch, przez co operacje odczytu i zapisu są wykonywane w jednej bazie danych.



Rysunek 7.2: Schemat komunikacji przy zastosowaniu wzorca CQRS

7.2.2. Zastosowanie DDD (Domain-Driven Design)

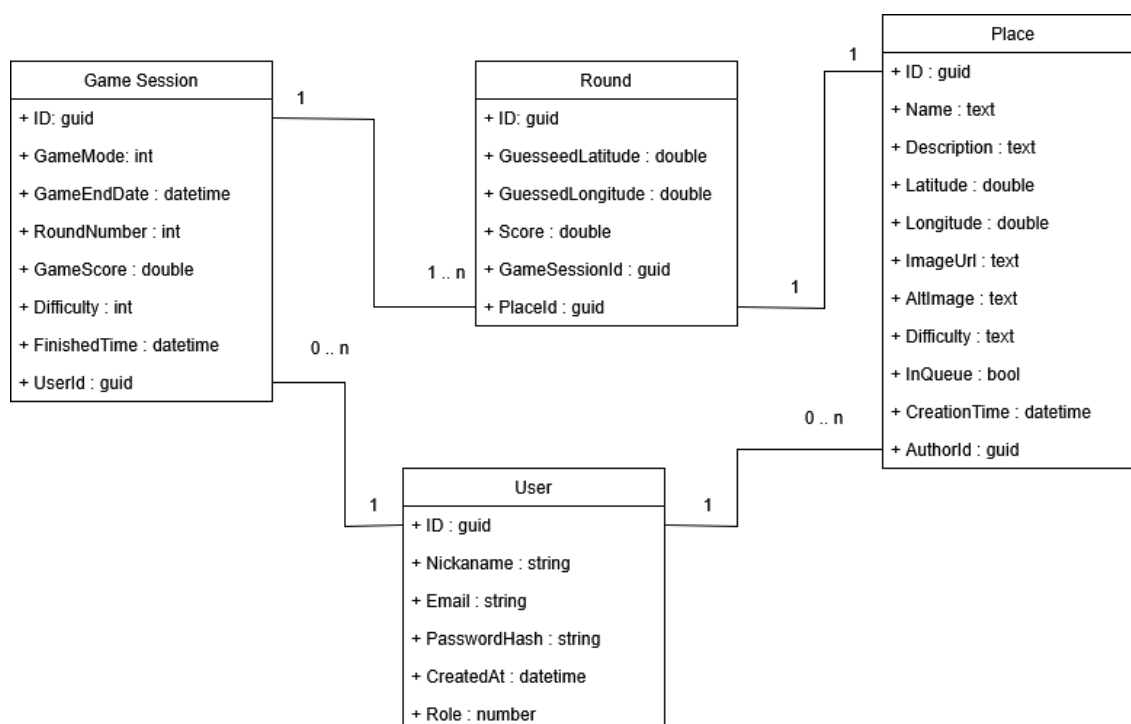
Model systemu został zaprojektowany zgodnie z podejściem Domain-Driven Design (DDD), które kładzie nacisk na zrozumienie i modelowanie domeny problemowej. W aplikacji definiujemy trzy główne warstwy DDD: warstwę domeny, warstwę aplikacji oraz warstwę infrastruktury. Każda z warstw znajduje się w osobnym module, co ułatwia zarządzanie kodem i jego rozwój. Dodatkowo w aplikacji wydzielony został czwarty moduł „API”, który odpowiada za komunikację z warstwą prezentacji, co zmniejsza złożoność warstwy aplikacji.

7.3. Baza danych

W aplikacji zastosowano relacyjną bazę danych, do przechowywania informacji o aktualnych i przeszłych sesjach gier, użytkownikach, lokalizacjach oraz wynikach. Zdecydowaliśmy się na wykorzystanie relacyjnej bazy danych ze względu na dużą ilość powiązań między encjami oraz potrzebę zapewnienia integralności danych między nimi.

7.3.1. Diagram ERD

Przedstawiony na rysunku 7.3 diagram ERD (Entity-Relationship Diagram) ilustruje strukturę bazy danych aplikacji UniGuesser, pokazując główne tabele oraz relacje między nimi. Użytkownik może posiadać wiele sesji gier, a każda sesja ma określoną liczbę rund, z których każda posiada przypisaną do siebie lokację. Ponadto każde miejsce może mieć swojego autora (użytkownika), który dodał je do bazy danych.



Rysunek 7.3: Diagram ERD (Entity-Relationship Diagram) dla bazy danych aplikacji UniGuesser

7.3.2. Opis głównych tabel i związków

- **Users (Użytkownicy)** – tabela przechowująca informacje o użytkownikach aplikacji.

Atrybut	Opis	Typ / Uwagi
ID	Unikalny identyfikator użytkownika.	Klucz główny
Nickname	Unikalna nazwa użytkownika.	Tekst
Email	Unikalny adres e-mail użytkownika.	Tekst
PasswordHash	Zaszyfrowane hasło użytkownika.	Tekst
CreatedAt	Data i czas utworzenia konta.	Data/Czas
Role	Rola użytkownika w systemie (np. użytkownik, administrator).	Enumeracja

Tabela 7.1: Tabela Users

GameSessions (Sesje gier) – tabela przechowująca informacje o aktywnych i zakończonych rozgrywkach.

Atrybut	Opis	Typ / Uwagi
ID	Unikalny identyfikator sesji gry.	Klucz główny
GameMode	Tryb gry w danej sesji.	Enumeracja
GameEndDate	Data, do której rozgrywka jest aktywna.	Data/Czas
RoundNumber	Aktualnie rozgrywana runda.	Liczba
GameScore	Łączna liczba punktów zdobytych.	Liczba całkowita
Difficulty	Poziom trudności sesji.	Enumeracja
FinishedTime	Data zakończenia rozgrywki.	Data/Czas
UserID	Identyfikator użytkownika.	Klucz obcy → Users

Tabela 7.2: Tabela GameSessions

Rounds (Rundy) – tabela przechowująca informacje o poszczególnych rundach w sesji gry.

Atrybut	Opis	Typ / Uwagi
ID	Unikalny identyfikator rundy.	Klucz główny
GuessedLatitude	Szerokość geograficzna wskazana przez użytkownika.	Liczba zmiennoprzecinkowa
GuessedLongitude	Długość geograficzna wskazana przez użytkownika.	Liczba zmiennoprzecinkowa
Score	Punkty zdobyte w rundzie.	Liczba całkowita
GameSessionID	Identyfikator sesji gry.	Klucz obcy → Game-Sessions
LocationID	Identyfikator lokacji.	Klucz obcy → Places

Tabela 7.3: Tabela Rounds

Places (Lokacje) – tabela przechowująca informacje o lokacjach dostępnych w grze.

Atrybut	Opis	Typ / Uwagi
ID	Unikalny identyfikator lokacji.	Klucz główny
Name	Nazwa lokacji.	Tekst
Description	Opis lokacji.	Tekst
Latitude	Szerokość geograficzna lokacji.	Liczba zmiennoprzecinkowa
Longitude	Długość geograficzna lokacji.	Liczba zmiennoprzecinkowa
Difficulty	Poziom trudności lokacji.	Enumeracja
ImageUrl	Adres URL obrazu lokacji.	Tekst
AltImage	Tekst alternatywny dla obrazu lokacji.	Tekst
CreationTime	Data i czas dodania lokacji.	Data/Czas
AuthorID	Identyfikator autora lokacji.	Klucz obcy → Users

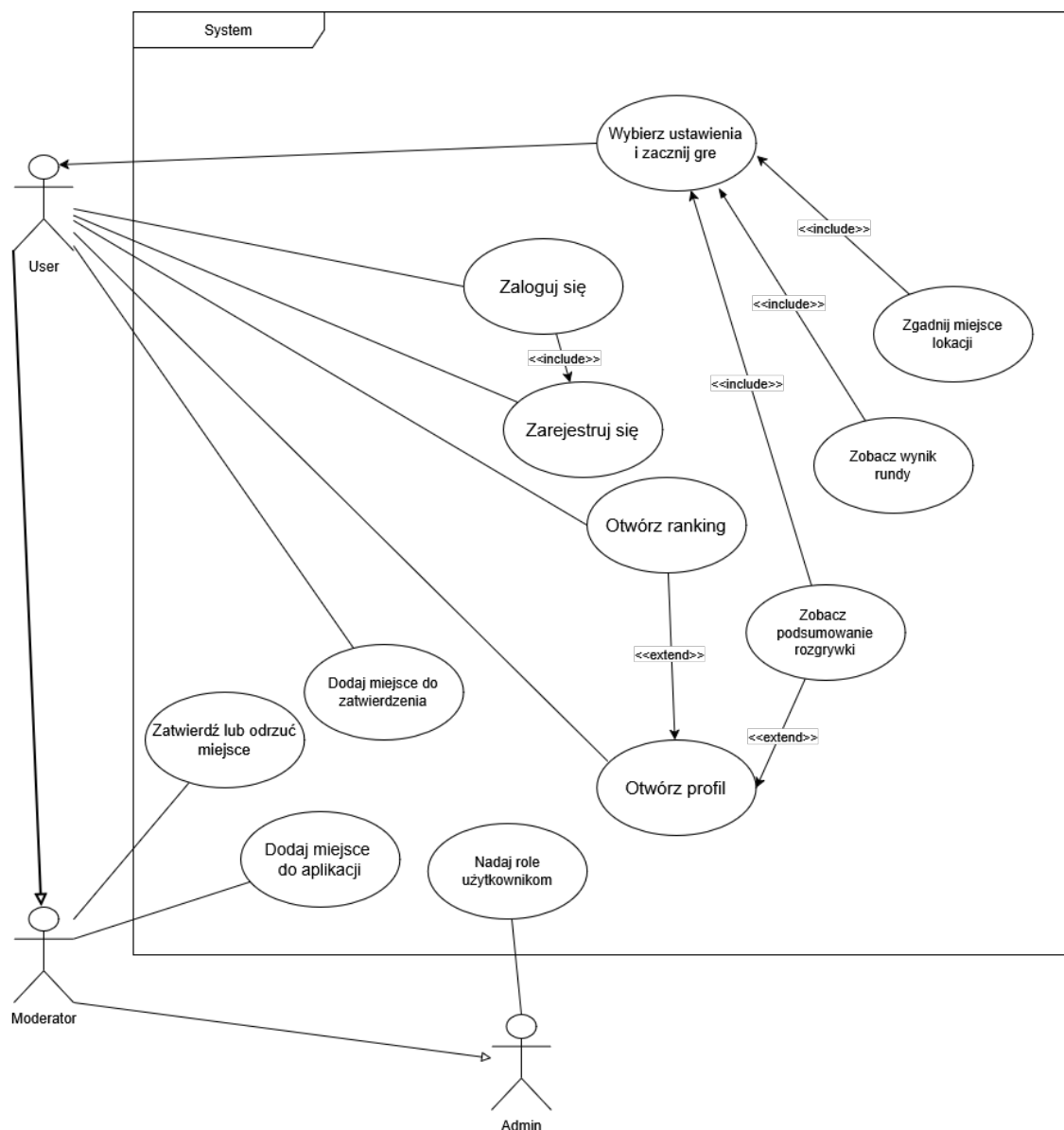
Tabela 7.4: Tabela Places

7.4. Analiza interakcji użytkownika z systemem

W rozdziale przedstawione zostaną możliwości i przebieg kluczowych funkcjonalności aplikacji

7.4.1. Przypadki użycia aplikacji

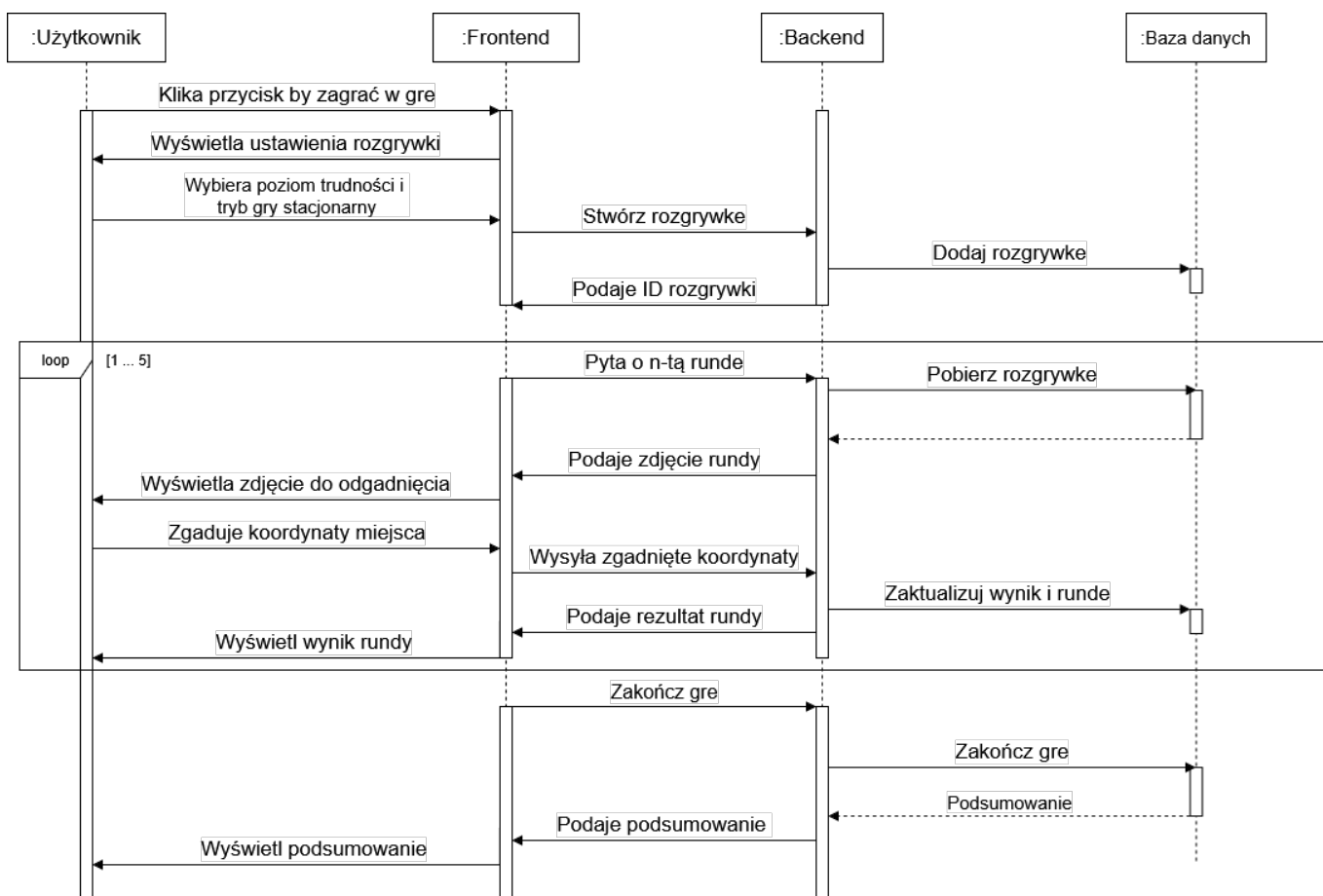
Przedstawiony na rysunku 7.4 diagram przypadków użycia ilustruje główne funkcjonalności aplikacji UniGuesser z perspektywy użytkownika moderatora i administratora. Użytkownik może rejestrować się i logować do systemu, rozpoczynać nowe sesje gier, uczestniczyć w rozgrywkach, przeglądać wyniki innych użytkowników, oraz proponować nowe lokacje. Moderator dodatkowo może zatwierdzać lub odrzucać proponowane lokacje, natomiast administrator ma uprawnienia do zarządzania użytkownikami.



Rysunek 7.4: Diagram przypadków użycia aplikacji UniGuesser

7.4.2. Diagram sekwencji rozgrywki

Diagram sekwencji przedstawiony na rysunku 7.5 ilustruje przebieg interakcji między użytkownikiem a systemem podczas typowej rozgrywki w aplikacji UniGuesser. Proces rozpoczyna się od momentu, gdy użytkownik uruchamia nową grę, poprzez kolejne rundy, aż do momentu zakończenia sesji i wyświetlenia wyników końcowych.



Rysunek 7.5: Diagram sekwencji przedstawiający przebieg rozgrywki w aplikacji UniGuesser

7.5. Estetyka i design

Wygląd aplikacji został zaprojektowany z myślą o prostocie, intuicyjności oraz atrakcyjności wizualnej. Poniżej przedstawiono kluczowe elementy designu interfejsu użytkownika.

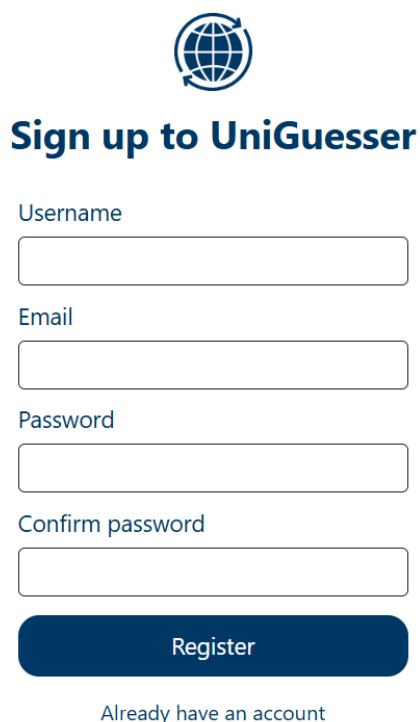
7.5.1. Ogólny projekt interfejsu

Design interfejsu użytkownika naszej aplikacji skupia się na minimalizmie i czytelności. Stosowane są przyjęte jako standardowe układy elementów, takie jak menu nawigacji w górnej części ekranu, co ułatwia poruszanie się po aplikacji, nawet dla nowych użytkowników. Podstrony są zaprojektowane tak, aby unikać nadmiaru informacji, co pozwala użytkownikom skupić się na kluczowych funkcjonalnościach. W trakcie rozgrywki centralnym elementem interfejsu jest interaktywna mapa, która umożliwia użytkownikom łatwe wskazywanie lokalizacji, a pozostałe elementy interfejsu są rozmieszczone w sposób intuicyjny wokół niej.

7.5.2. Kolorystyka

Aplikacja wykorzystuje dwukolorową paletę barw, opartą na odcieniach białego i ciemnoniebieskiego, inspirowaną kolorystyką Politechniki Gdańskiej. Biały kolor dominuje w tle, co zapewnia czystość i przejrzystość interfejsu, natomiast niebieskie akcenty są używane do wyróżnienia kluczowych elementów, takich jak przyciski, nagłówki czy linki. Przykładowe zastosowanie palety kolorów przedstawia rysunek 7.6. Taka kolorystyka zapewnia odpowiedni kontrast i ułatwia czytelność tekstu, zwłaszcza przy ko-

rzystaniu z aplikacji w różnych warunkach oświetleniowych, co jest szczególnie istotne ze względu na terenowy charakter gry.



The image shows a registration form for the UniGuesser application. At the top center is a logo consisting of a globe with a circular arrow around it. Below the logo is the title "Sign up to UniGuesser" in a bold, dark blue font. The form contains four input fields, each with a label above it: "Username", "Email", "Password", and "Confirm password". All labels and the "Register" button text are in a dark blue font. The "Register" button is a solid dark blue rectangle with rounded corners. Below the button is a link that says "Already have an account" in a smaller, lighter blue font.

Rysunek 7.6: Przykładowy ekran rejestracji użytkownika w aplikacji UniGuesser, ilustrujący zastosowaną kolorystykę

7.5.3. Typografia

Aplikacja wykorzystuje czcionkę Roboto, która jest nowoczesna, czytelna i estetyczna. Czcionka ta jest szeroko stosowana w projektach webowych ze względu na swoją uniwersalność i dobrą czytelność na ekranach o różnych rozdzielczościach.

7.5.4. Ikony

W interfejsie użytkownika wykorzystano zestaw prostych i intuicyjnych ikon, pochodzących z biblioteki Google Icons. Ikony te są spójne stylistycznie i pomagają użytkownikom szybko zidentyfikować funkcje oraz akcje dostępne w aplikacji. Dzięki zastosowaniu powszechnie rozpoznawalnych symboli, nawigacja staje się bardziej intuicyjna, co przekłada się na lepsze doświadczenie użytkownika.

7.5.5. Responsywność

Aplikacja została zaprojektowana z myślą o responsywności, co oznacza, że dostosowuje się do różnych rozmiarów ekranów i urządzeń. Dzięki temu użytkownicy mogą korzystać z aplikacji zarówno na komputerach stacjonarnych, jak i urządzeniach mobilnych, bez utraty funkcjonalności czy estetyki. Elementy interfejsu, takie jak przyciski, formularze czy mapy, automatycznie dostosowują swoje rozmiary i układ w zależności od dostępnej przestrzeni ekranowej. Na mniejszych ekranach, takich jak smartfony, menu nawigacyjne jest ukryte za ikoną hamburgera, co pozwala zaoszczędzić miejsce i utrzymać przejrzystość interfejsu. Ponadto, kluczowe funkcje pozostają łatwo dostępne, a tekst i grafiki są skalowane

odpowiednio do rozdzielczości ekranu, zapewniając optymalne doświadczenie użytkownika niezależnie od urządzenia.

8. IMPLEMENTACJA

9. TESTOWANIE

10. PODSUMOWANIE

11. MOŻLIWOŚĆ DALSZEGO ROZWOJU

WYKAZ RYSUNKÓW

7.1	Uproszczony schemat architektury systemu UniGuesser – ogólny podział na warstwy . . .	20
7.2	Schemat komunikacji przy zastosowaniu wzorca CQRS	20
7.3	Diagram ERD (Entity-Relationship Diagram) dla bazy danych aplikacji UniGuesser . . .	21
7.4	Diagram przypadków użycia aplikacji UniGuesser	24
7.5	Diagram sekwencji przedstawiający przebieg rozgrywki w aplikacji UniGuesser	25
7.6	Przykładowy ekran rejestracji użytkownika w aplikacji UniGuesser, ilustrujący zastosowaną kolorystykę	26

WYKAZ TABEL

7.1	Tabela Users	22
7.2	Tabela GameSessions	22
7.3	Tabela Rounds	23
7.4	Tabela Places	23